



O-RAN Work Group 11 (Security Work Group)

Security Protocols Specifications

Copyright © 2025 by the O-RAN ALLIANCE e.V.

The copying or incorporation into any other work of part or all of the material available in this specification in any form without the prior written permission of O-RAN ALLIANCE e.V. is prohibited, save that you may print or download extracts of the material of this specification for your personal use, or copy the material of this specification for the purpose of sending to individual third parties for their information provided that you acknowledge O-RAN ALLIANCE as the source of the material and that you inform the third party that these conditions apply to them and that they must comply with them.

O-RAN ALLIANCE e.V., Buschkauler Weg 27, 53347 Alfter, Germany
Register of Associations, Bonn VR 11238, VAT ID DE321720189

Contents

List of figures	4
List of tables	4
Foreword.....	5
Modal verbs terminology	5
1 Scope	6
2 References	6
2.1 Normative references	6
2.2 Informative references	10
3 Definition of terms, symbols and abbreviations	11
3.1 Terms	11
3.2 Symbols	11
3.3 Abbreviations	11
4 Security protocols specifications for O-RAN compliant implementation	12
4.1 SSH	12
4.1.1 General requirements	12
4.1.2 Required ciphers.....	12
4.2 TLS	13
4.2.1 General Requirements	13
4.2.2 TLS Protocol Profile	14
4.2.3 TLS Certificate Profile	14
4.3 Support NETCONF over secure Transport.....	15
4.4 DTLS	15
4.4.1 General requirements	15
4.4.2 DTLS Protocol Profile	15
4.4.3 Certificate Profile	15
4.5 IPsec	15
4.5.1 General Requirements	15
4.5.2 Parallel usage of IPsec and other secure transport protocols	17
4.5.3 Responder mode and Initiator/Responder mode support	17
4.6 CMPv2	17
4.7 OAuth 2.0.....	17
4.7.1 Overview	17
4.7.2 Basic Parameterization.....	17
4.7.3 OAuth2.0 Access token API for O-RAN Elements and interfaces.....	20
4.8 LDAP	25
4.8.1 General requirements	25
4.9 MACsec	25
4.9.1 General requirements	25
4.9.2 MACsec Profile.....	25
5 Cryptographic operations	26
6 Secure File Transfer protocols	29
6.1 General	29
6.2 SFTP	29
6.2.1 General Requirements	29
6.3 FTPES	29
6.3.1 General Requirements	29
6.4 HTTPS	29
6.4.1 General Requirements	29
Annex A (normative): OpenAPI specifications	30
A.1 OAuth2.0 API	30
A.1.1 Introduction	30
A.1.2 OAuth 2.0 Access Token API	30



Annex B (informative): Example of TLS entity Certificate Profile for NETCONF authentication	32
Annex (informative): Change history/Change request (history)	34

List of figures

Figure 4.7-1: Access Token request process.....	18
Figure 4.7-2: Service access request based on token verification	19
Figure 4.7-3: Resource structure for access token	20
Figure 4.7-4: Access Token Request and Response	21
Figure 4.7-5: Service request using access token	24

List of tables

Table 4.7-1: Resources and methods overview of the Access Token API	20
Table 4.7-2: Data structures supported by the POST request body on this resource.....	21
Table 4.7-3: Data structures supported by the POST Response Body on this resource.	22
Table 4.7-4: Headers supported by the 200 Response Code on this resource.	22
Table 4.7-5: Definition of type AccessTokenReq	22
Table 4.7-6: Definition of type AccessTokenResp.....	23
Table 4.7-7: Definition of type AccessTokenErr.....	23
Table 5-1: Cryptographic Operations for O-RAN	27
Table B-1	32

Foreword

This Technical Specification (TS) has been produced by WG11 of the O-RAN Alliance.

The content of the present document is subject to continuing work within O-RAN and may change following formal O-RAN approval. Should the O-RAN Alliance modify the contents of the present document, it will be re-released by O-RAN with an identifying change of version date and an increase in version number as follows:

version xx.yy.zz

where:

- xx: the first digit-group is incremented for all changes of substance, i.e. technical enhancements, corrections, updates, etc. (the initial approved document will have xx=01). Always 2 digits with leading zero if needed.
- yy: the second digit-group is incremented when editorial only changes have been incorporated in the document. Always 2 digits with leading zero if needed.
- zz: the third digit-group included only in working versions of the document indicating incremental changes during the editing process. External versions never include the third digit-group. Always 2 digits with leading zero if needed.

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the O-RAN Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in O-RAN deliverables except when used in direct citation.

1 Scope

The present document specifies security protocols as to be used for O-RAN compliant implementation.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document, a non-specific reference implicitly refers to the latest version of that document in Release 18, or the latest 3GPP release prior to Release 18 that includes that document.

NOTE 1: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

NOTE 2: All specifications issued by IETF referenced in the present document are valid in the latest version which the IETF declares as valid. Any updates to valid RFCs have to be considered and implemented, if applicable.

- [1] Void.
- [2] IETF RFC 4252: "The Secure Shell (SSH) Authentication Protocol".
- [3] Void.
- [4] Void.
- [5] IANA: "Secure Shell (SSH) Protocol Parameters".
- [6] O-RAN ALLIANCE TS: "O-RAN Management Plane Specification".
- [7] Void.
- [8] IANA: "Transport Layer Security (TLS) Parameters".
- [9] Void.
- [10] O-RAN ALLIANCE TS: "O-RAN Operations and Maintenance Interface Specification" .
- [11] IETF RFC 6668: "SHA-2 Data Integrity Verification for the Secure Shell (SSH) Transport Layer Protocol".
- [12] IETF RFC 8268: "More Modular Exponentiation (MODP) Diffie-Hellman (DH) Key Exchange (KEX) Groups for Secure Shell (SSH)".
- [13] IETF RFC 8308: "Extension Negotiation in the Secure Shell (SSH) Protocol".
- [14] IETF RFC 8332: "Use of RSA Keys with SHA-256 and SHA-512 in the Secure Shell (SSH) Protocol".
- [15] IETF RFC 8709: "Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol".
- [16] Void.
- [17] Void.
- [18] IETF RFC 8446: "The Transport Layer Security Protocol (TLS) v1.3".
- [19] Void.
- [20] 3GPP TS 33.210: "Network Domain Security (NDS); IP network layer security".

- [21] Void.
- [22] IETF RFC 6066: "Transport Layer Security (TLS) Extensions: Extension Definitions".
- [23] Void.
- [24] IETF RFC 6083: "Datagram Transport Layer Security (DTLS) for Stream Control Transmission Protocol (SCTP)".
- [25] Void.
- [26] 3GPP TS 33.310: Network Domain Security (NDS); Authentication Framework (AF)".
- [27] IETF RFC 4301: "Security Architecture for the Internet Protocol".
- [28] 3GPP TS 33.401: "3GPP System Architecture Evolution (SAE); Security architecture".
- [29] 3GPP TS 33.501: "Security architecture and procedures for 5G system".
- [30] Void.
- [31] IETF RFC 4303: "IP Encapsulating Security Payload (ESP)".
- [32] Void.
- [33] Void.
- [34] IETF RFC 7296: "Internet Key Exchange Protocol Version 2 (IKEv2)".
- [35] IETF RFC 6749: "The OAuth 2.0 Authorization framework".
- [36] IETF RFC 7519: "JSON Web Token (JWT)".
- [37] IETF RFC 7515: "JSON Web Signature (JWS)".
- [38] IETF RFC 4210: "Internet X.509 Public Key Infrastructure Certificate Management Protocol (CMP)".
- [39] IETF RFC 4211: "Internet X.509 Public Key Infrastructure Certificate Request Message Format".
- [40] NIST: "Cryptographic Standards and Guidelines".
- [41] Void.
- [42] NIST: "Digital Signature Standard (DSS) (FIPS PUB 186)".
- [43] Void.
- [44] NIST: "Advanced Encryption Standard (AES) (FIPS PUB 197)".
- [45] Void.
- [46] Void.
- [47] NIST: "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Function (FIPS PUB 202)".
- [48] NIST SP 800-131A Rev. 2: "Transitioning the Use of Cryptographic Algorithms and Key Lengths".
- [49] Void.
- [50] IETF RFC 8017: "PKCS #1: RSA Cryptography Specifications Version 2.2".
- [51] Void.
- [52] Void.
- [53] Void.
- [54] Void.
- [55] IETF RFC 7633: "X.509v3 Transport Layer Security (TLS) Feature Extension".

- [56] CA/Browser Forum: "Baseline Requirements for the Issuance and Management of Publicly-Trusted Certificates".
- [57] Void.
- [58] Void.
- [59] Void.
- [60] IETF RFC 7093: "Additional Methods for Generating Key Identifiers Values".
- [61] Void.
- [62] IETF RFC 6960: "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP".
- [63] IETF RFC 3647: "Internet X.509 Public Key Infrastructure Certificate Policy and Certification Practices Framework".
- [64] IETF RFC 5280: "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile".
- [65] SFTP: "SFTP Standards".
- [66] IETF RFC 4217: "Securing FTP with TLS (FTPES)".
- [67] 3GPP TS 28.532: "Management and orchestration; Generic management services".
- [68] 3GPP TS 28.537: "Management and orchestration; Management capabilities".
- [69] IETF RFC 9110: "HTTP Semantics (HTTPS)".
- [70] IETF RFC 4253: "The Secure Shell (SSH) Transport Layer Protocol".
- [71] IETF RFC 9325: "Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)".
- [72] IETF RFC 3279: "Algorithms and Identifiers for the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile".
- [73] ITU-T Recommendation X.509: "Information technology - Open Systems Interconnection - The Directory: Public-key and attribute certificate frameworks".
- [74] IETF RFC 4511: "Lightweight Directory Access Protocol (LDAP): The Protocol".
- [75] IETF RFC 4513: "Lightweight Directory Access Protocol (LDAP): Authentication Methods and Security Mechanisms".
- [76] IETF RFC 9147: "The Datagram Transport Layer Security (DTLS) Protocol Version 1.3".
- [77] 3GPP TS 29.501: "5G System; Principles and Guidelines for Services Definition".
- [78] IETF RFC 6750: "The OAuth 2.0 Authorization Framework: Bearer Token Usage".
- [79] IEEE Std 802.1AE-2018: "IEEE Standard for Local and metropolitan area networks — Media Access Control (MAC) Security".
- [80] IETF RFC 9519: "Update to the IANA SSH Protocol Parameters Registry Requirements".
- [81] IETF RFC 7427: "Signature Authentication in the Internet Key Exchange Version 2 (IKEv2)".
- [82] IETF RFC 7670: "Generic Raw Public-Key Support for IKEv2".
- [83] IETF RFC 8247: "Algorithm Implementation Requirements and Usage Guidance for the Internet Key Exchange Protocol Version 2 (IKEv2)".
- [84] IETF RFC 8983: "Internet Key Exchange Protocol Version 2 (IKEv2) Notification Status Types for IPv4/IPv6 Coexistence".

- [85] IETF RFC 9370: "Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2)".
- [86] IETF RFC 5656: "Elliptic Curve Algorithm Integration in the Secure Shell Transport Layer"
- [87] IETF RFC 5647: "AES Galois Counter Mode for the Secure Shell Transport Layer Protocol"
- [88] IETF RFC 4344: "The Secure Shell (SSH) Transport Layer Encryption Modes"
- [89] IETF RFC 4419: "Diffie-Hellman Group Exchange for the Secure Shell (SSH) Transport Layer Protocol"
- [90] IETF RFC 8270. "Increase the Secure Shell Minimum Recommended Diffie-Hellman Modulus Size to 2048 Bits"
- [91] IETF RFC 8731, "Secure Shell (SSH) Key Exchange Method Using Curve25519 and Curve448"
- [92] IETF RFC 4418, "UMAC: Message Authentication Code using Universal Hashing"
- [93] IETF RFC 8725: "JSON Web Token Best Current Practices".
- [94] IETF RFC 7797: "JSON Web Signature (JWS) Unencoded Payload Option".
- [95] IETF RFC 9480: "Certificate Management Protocol (CMP) Updates".
- [96] IETF RFC 9481: "Certificate Management Protocol (CMP) Algorithms".
- [97] IETF RFC 9045: "Algorithm Requirements Update to the Internet X.509 Public Key Infrastructure Certificate Request Message Format (CRMF)".
- [98] IETF RFC 9525: "Service Identity in TLS".
- [99] IETF RFC 9654: "Online Certificate Status Protocol (OCSP) Nonce Extension".
- [100] IETF RFC 6818: "Updates to the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile".
- [101] IETF RFC 9549: "Internationalization Updates to RFC 5280".
- [102] IETF RFC 9598: "Internationalized Email Addresses in X.509 Certificates".
- [103] IETF RFC 9608: "No Revocation Available for X.509 Public Key Certificates".
- [104] IETF RFC 9618: "Updates to X.509 Policy Validation".
- [105] IETF RFC 6668: "SHA-2 Data Integrity Verification for the Secure Shell (SSH) Transport Layer Protocol".
- [106] IETF RFC 8268: "More Modular Exponentiation (MODP) Diffie-Hellman (DH) Key Exchange (KEX) Groups for Secure Shell (SSH)".
- [107] IETF RFC 8308: "Extension Negotiation in the Secure Shell (SSH) Protocol".
- [108] IETF RFC 8332: "Use of RSA Keys with SHA-256 and SHA-512 in the Secure Shell (SSH) Protocol".
- [109] IETF RFC 8709: "Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol".
- [110] IETF RFC 8758: "Deprecating RC4 in Secure Shell (SSH)".
- [111] IETF RFC 9142: "Key Exchange (KEX) Method Updates and Recommendations for Secure Shell (SSH)".
- [112] IETF RFC 8996: "Deprecating TLS 1.0 and TLS 1.1".
- [113] IETF RFC 4055: "Additional Algorithms and Identifiers for RSA Cryptography for use in the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile".
- [114] IETF RFC 5756: "Updates for RSAES-OAEP and RSASSA-PSS Algorithm Parameters".
- [115] IETF RFC 5480: "Elliptic Curve Cryptography Subject Public Key Information".

- [116] IETF RFC 8813: "Clarifications for Elliptic Curve Cryptography Subject Public Key Information".
- [117] IETF RFC 5758: "Internet X.509 Public Key Infrastructure: Additional Algorithms and Identifiers for DSA and ECDSA".
- [118] IETF RFC 8692: "Internet X.509 Public Key Infrastructure: Additional Algorithm Identifiers for RSASSA-PSS and ECDSA Using SHAKEs".
- [119] NIST: "Secure Hash Standard (SHS) " (FIPS PUB 180-4)
- [120] IETF RFC 9113: "Hypertext Transfer Protocol Version 2 (HTTP/2)".

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document, a non-specific reference implicitly refers to the latest version of that document in Release 18, or the latest 3GPP release prior to Release 18 that includes that document.

NOTE 1: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long term validity.

The following referenced documents are not necessary for the application of the present document but they assist the user with regard to a particular subject area.

NOTE 2: All specifications issued by IETF referenced in the present document are valid in the latest version which the IETF declares as valid. Any updates to valid RFCs have to be considered and implemented, if applicable.

- [i.1] 3GPP TR 21.905: "Vocabulary for 3GPP Specifications".
- [i.2] Void.
- [i.3] Void.
- [i.4] NIST SP 800-52 Rev.2: "Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations".
- [i.5] Void.
- [i.6] BSI TR-02102-1: "Cryptographic Mechanisms: Recommendations and Key Lengths".
- [i.7] NIST SP 800-38A: "Recommendation for Block Cipher Modes of Operation: Methods and Techniques".
- [i.8] NIST SP 800-186: "Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters".
- [i.9] NIST SP 800-57 Part 1 Rev. 5: "Recommendation for Key Management: Part 1 – General".
- [i.10] Void.
- [i.11] IETF RFC 7589: "Using the NETCONF Protocol over Transport Layer Security (TLS) with Mutual X.509 Authentication".
- [i.12] NIST SP 800-90A: "Recommendation for Random Number Generation Using Deterministic Random Bit Generators".
- [i.13] NIST SP 800-90B: "Recommendation for the Entropy Sources Used for Random Bit Generation".
- [i.14] NIST SP 800-90C: "Recommendation for Random Bit Generator (RBG) Constructions".
- [i.15] NIST FIPS 140-3: "Security Requirements for Cryptographic Modules".
- [i.16] BSI: "Documentation and Analysis of the Linux Random Number Generator"

3 Definition of terms, symbols and abbreviations

3.1 Terms

Void

3.2 Symbols

Void

3.3 Abbreviations

For the purposes of the present document, the abbreviations given in 3GPP TR 21.905 [i.1] and the following apply. An abbreviation defined in the present document takes precedence over the definition of the same abbreviation, if any, in 3GPP TR 21.905 [i.1].

0-RTT	Zero Round Trip Time
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
CA	Certification Authority
CMP	Certificate Management Protocol
CSPRNG	Cryptographically secure pseudo-random number generator
DH	Diffie–Hellman
DHE	Diffie–Hellman Ephemeral
DPD	Dead Peer Detection
DRBG	Deterministic Random Bit Generators
DTLS	Datagram Transport Layer Security
ECDHE	Elliptic Curve Diffie–Hellman Ephemeral
ESP	Encapsulating Security Payload
ETM	Encrypt-then-MAC
FTPES	Explicit SSL File Transfer Protocol
GCM	Galois Counter Mode
HTTPS	Hypertext Transfer Protocol Secure
IKE	Internet Key Exchange
IPsec	Internet Protocol Security
NAT	Network Address Translation
NE	Network Element
NETCONF	Network Configuration Protocol

PFS	Perfect Forward Secrecy
PKI	Public Key Infrastructure
PSK	Pre-Shared Key
RA	Registration Authority
RNG	Random Number Generation
SA	Security Association
SFTP	SSH File Transfer Protocol
SHA	Secure Hash Algorithm
SPD	Security Policy Database
SSH	Secure Shell
TLS	Transport Layer Security

4 Security protocols specifications for O-RAN compliant implementation

4.1 SSH

4.1.1 General requirements

O-RAN interfaces that implement authentication, confidentiality and integrity using SSH shall:

- support SSHv2 as defined in IETF RFC 4252 [2], IETF RFC 4253 [70], IETF RFC 6668 [105], IETF RFC 8268 [106], IETF RFC 8308 [107], IETF RFC 9519 [80], IETF RFC 8332 [108], IETF RFC 8709 [109], IETF RFC 8758 [110], and IETF RFC 9142 [111];
- not support SSHv1
- disable by default cryptographically insecure ciphers as specified in clauses 4.1.2.1, 4.1.2.3, and 4.1.2.4;
- enable an O-RAN deployer to use standard SSH configurations to select the algorithms offered for negotiation;
- enable remote shell access only if required by the interface. If remote shell access is enabled, disable remote login as root or equivalent highest privileged user. Such access shall be limited to the local system console only. Root user shall not be allowed to login to the system remotely.

Entities providing O-RAN components that support SSH for authentication, confidentiality or integrity shall:

- stay current with SSH [5];
- provide an upgrade path for changes to the SSH protocol and ciphers [5].

4.1.2 Required ciphers

4.1.2.0 Introduction

O-RAN requires the following ciphers when using SSH. For more information see [2], [5], [11], [12], [13], [14], [15] and [80]. See Security clause 5.4 of the O-RAN Working Group 4 Management Plane Specification [6] for the M-plane mandated SSH ciphers.

4.1.2.1 Key agreement

NOTE: the present document uses the IANA cipher naming convention [5].

- Shall be supported:
 - a. ecdsa-sha2-nistp256 as defined in IETF RFC 5656 [86]
 - b. ecdsa-sha2-nistp384 as defined in IETF RFC 5656 [86]
 - c. ecdsa-sha2-nistp521 as defined in IETF RFC 5656 [86]
 - d. ssh-ed25519 as defined in IETF RFC 8709 [15]
- Should be supported:
 - a. ssh-ed448 as defined in IETF RFC 8709 [15]
- Shall not be supported:
 - a. ssh-rsa
 - b. ssh-dss

4.1.2.2 Symmetric algorithms for encrypting transferred data

- Shall be supported
 - a. aes256-gcm as defined in IETF RFC 5647 [87]
 - b. aes128-gcm as defined in IETF RFC 5647 [87]
 - c. aes256-ctr as defined in IETF RFC 4344 [88]
 - d. aes192-ctr as defined in IETF RFC 4344 [88]
 - e. aes128-ctr as defined in IETF RFC 4344 [88]

4.1.2.3 Key exchange algorithms (KexAlgorithms)

- Shall be supported
 - a. ecdh-sha2-nistp521 as defined in IETF RFC 5656 [86]
 - b. ecdh-sha2-nistp384 as defined in IETF RFC 5656 [86]
 - c. ecdh-sha2-nistp256 as defined in IETF RFC 5656 [86]
 - d. diffie-hellman-group-exchange-sha256 as defined in IETF RFC 4419 [89], updated by IETF RFC 8270 [90]
 - e. curve25519-sha256 as defined in IETF RFC 8731 [91]
 - f. diffie-hellman-group14-sha256 as defined in IETF RFC 8268 [12]
- Shall not be supported:
 - a. diffie-hellman-group1-sha1
 - b. diffie-hellman-group-exchange-sha1
 - c. gss-group1-sha1-*
 - d. gss-group14-sha1-*
 - e. gss-gex-sha1-*
 - f. rsa1024-sha1

4.1.2.4 Message Authentication Codes (MACs)

- Shall be supported
 - a. hmac-sha2-512-etm (see NOTE)
 - b. hmac-sha2-512 as defined in IETF RFC 6668 [11]
 - c. hmac-sha2-256-etm (see NOTE)
 - d. hmac-sha2-256 as defined in IETF RFC 6668 [11]
 - e. umac-128 as defined in IETF RFC 4418 [92]
- Shall not be supported:
 - a. hmac-sha1

NOTE: ETM (Encrypt-then-MAC) does not have an SSH specification yet is a widely used and recommended practice within the SSHv2 protocol, and broadly supported in available implementations, due to its security-related advantages.

4.2 TLS

4.2.1 General Requirements

O-RAN interfaces that implement one-way authentication, mutual authentication, confidentiality, integrity, and replay protection using Transport Layer Security (TLS) shall be compliant with [18], allow negotiation of both TLS 1.2 [18] and TLS 1.3 [18], and not allow negotiation to any version of SSL or any version of TLS below 1.2. Further, O-RAN interfaces using

TLS may allow TLS 1.2 to be disabled by configuration ([i.4], section 3.1). O-RAN interfaces that use a pre-shared key (PSK) should not send data on the first flight (early or 0-RTT data) because the security properties for these data are weaker than those for other kinds of TLS data and can lead to replay attacks.

NOTE 1: In one-way TLS the server sends a Certificate message, a CertificateVerify message, and a Finished message to the client so that the client can authenticate the server by validating the server's certificate and possession of the private key associated with the certificate. In mutual TLS (mTLS) the server sends a CertificateRequest message indicating that the client must authenticate itself with a certificate. The server then sends a Certificate message, a CertificateVerify message, and a Finished message to the client. Upon receiving the server's messages, the client responds with its Authentication messages, namely Certificate and CertificateVerify, and Finished. For full details of the authentication steps in TLS see [18].

NOTE 2: [18] added a zero round-trip time (0-RTT) mode, saving a round trip at connection setup for some application data, at the cost of replay protection. When clients and servers share a PSK, TLS 1.3 allows clients to send data on the first flight ("early data"). The client uses the PSK to authenticate the server and to encrypt the early data. See Section 2.3 of [18] for more information.

NOTE 3: See Appendix E.5 of [18] for a description of potential attacks, and Section 8 of [18] for mechanisms which the server can use to limit the impact of replay when using 0-RTT data.

See clause 5.4 of the O-RAN Working Group 4 Management Plane Specification for the Open Fronthaul M-plane mandated TLS requirements [6].

4.2.1.1 Void

Void.

4.2.2 TLS Protocol Profile

TLS 1.2 used on an O-RAN specified interface shall support the TLS 1.2 profiles as defined by 3GPP TS 33.210 [20], clause 6.2.3.

The Open Fronthaul has the following exception to 3GPP TS 33.210 [20], clause 6.2.3:

- TLS 1.2 used on the Open Fronthaul interface may support cipher suites with AEAD (e.g. GCM) and PFS (e.g. ECDHE, DHE).

TLS 1.3 used on an O-RAN specified interface shall support the TLS 1.3 profiles as defined by 3GPP TS 33.210 [20] clause 6.2.2.

Use of a cipher suite in TLS shall be configurable.

Additional cipher suites recommended by IANA [8] may be supported.

The TLS cipher suites in RFC 9113 [120], Appendix A, shall be disabled or not supported.

NOTE: The vendor and operator need to be prepared to replace integrity and/or ciphering algorithms if the current algorithm in use is compromised or deprecated.

4.2.3 TLS Certificate Profile

The present clause addresses the certificate profile requirements for the TLS entities that may behave either as client, server, or both. The profile of the certificates to be used with TLS is provided in this clause is based on requirements from 3GPP TS 33.310 [26]. The TLS entity certificate profile shall be applied to all nodes and interfaces that use the TLS protocol for secured communication in the O-RAN network.

- The common rules for all certificates defined in 3GPP- TS 33.310 [26], clause 6.1.1 shall apply.

- The TLS entity certificate profile defined in 3GPP TS 33.310 [26], 6.1.3a shall apply.

NOTE: Annex B provides an example of a TLS entity certificate profile used for NETCONF authentication.

4.3 Support NETCONF over secure Transport

TLS requirements for use with NETCONF on the O1 interface [10] are specified in clause 4.2.

TLS requirements for use with NETCONF on the Open Fronthaul M-plane are specified in clause 5.4 of the O-RAN WG4 Management Plane Specification [6].

4.4 DTLS

4.4.1 General requirements

O-RAN interfaces that implement mutual authentication, integrity protection, replay protection and confidentiality protection using Datagram Transport Layer Security (DTLS) shall:

- support DTLS 1.2 [76];
- support DTLS for Stream Control Transmission Protocol [24];

and should support DTLS 1.3 [76].

DTLS 1.0 shall not be supported.

NOTE: IETF RFC 6083 [24] specifies a user message limit of approximately 16k bytes, which does not align with the unlimited user message size that exists when SCTP is used without DTLS. There could be applications messages exceeding the limit, preventing the use of DTLS over SCTP, so enforcing unsecured SCTP.

4.4.2 DTLS Protocol Profile

For DTLS 1.2, all TLS 1.2 related provisions in clause 4.2.2 shall apply.

For DTLS 1.3, all TLS 1.3 related provisions in clause 4.2.2 shall apply.

4.4.3 Certificate Profile

Certificate profile defined in clause 0 shall apply.

4.5 IPsec

4.5.1 General Requirements

4.5.1.1 Supported IPsec capabilities

O-RAN interfaces that implement authentication, confidentiality and integrity using IPsec shall support IPsec according to IETF RFC 4301 [27]. The supported IPsec capabilities in this clause follow 3GPP TS 33.210 [20] for interworking purposes and further apply requirements given in [26], [28], and [29].

Services that shall be supported (see also 3GPP TS 33.501 [29], clause 9.8.2):

- confidentiality, can be enabled/disabled (via ENCR_NULL);
- integrity protection;
- data origin authentication;
- replay protection, can be enabled/disabled;
- extended sequence numbers, can be enabled/disabled.

Protocol that shall be supported:

- ESP (IETF RFC 4303 [31], as profiled by 3GPP TS 33.210 [20]);
- IPsec mode: Tunnel mode;
- for encrypted packets the DSCP (Differentiated Services Code Point) value of the inner packet shall be copied to the outer packet;
- NAT (Network Address Translation) traversal.

ESP (Encapsulating Security Payload) encryption transforms shall be supported (including authenticated encryption transforms, see also 3GPP TS 33.210 [20], clause 5.3.3).

ESP authentication transforms shall be supported according to [20], clause 5.3.4.

IKE endpoint Identification that shall be supported, according to 3GPP TS 33.310 [26], clause 6.2.1b:

- IP address;
- Fully Qualified domain name (FQDN) (if DNS is supported).

Authentication that shall be supported:

- X.509v3 digital certificates provided by a Certificate Authority (CA) solution

Authentication that may be supported:

- Pre-shared Keys

NOTE: Use of Pre-shared Keys is not recommended.

Key exchanges that shall be supported:

- IKEv2 as defined in IETF RFC 7296 [34], IETF RFC 7427 [81], IETF RFC 7670 [82], IETF RFC 8247 [83], IETF RFC 8983 [84], IETF RFC 9370 [85], profile as described in 3GPP TS 33.310 [26];
- certificate-based authentication according to 3GPP TS 33.310 [26];
- certificates according to the profile described by 3GPP TS 33.310 [26];
- Diffie-Hellman (DH) groups:
 - DH group 19;
 - DH group 20 (optional).

Implementations may follow recommendations regarding minimum key lengths proposed in BSI TR-02102-1 [i.6] .

IKEv1 shall not be supported.

Security Association multiplicity that shall be supported:

- multiple IKE SAs (multiple IPsec tunnels);
- multiple IPsec SAs;
- multiple IPsec SAs per IPsec tunnel.

IKE SA(s) and IPsec SA(s) shall be regularly re-established.

When the full sequence number space of an IPsec SA(s) is used, the transmission for that SA shall stop.

Dead-Peer-Detection (DPD) shall be supported as defined in [34].

Each node shall support a traffic narrowing function for the traffic selector ([34]). If the traffic selector notified from the peer (e.g. Security GW or neighbouring node) is wider than the Local/Remote Address range in the SPD (Security Policy Database) information set on the node side, the peers set the traffic selector values to the narrower range.

4.5.2 Parallel usage of IPsec and other secure transport protocols

Implementations SHALL support IPsec configuration with one or more dedicated connections in parallel with other secure transport protocols.

EXAMPLE: It should be possible to run SSH or TLS connections within an IPsec tunnel or in parallel to IPsec tunnel(s).

4.5.3 Responder mode and Initiator/Responder mode support

Implementations SHALL support a configurable option per IKE Security Association to use "Responder mode" instead of "Responder/Initiator mode".

Responder mode is applied to IKE SA establishment only. The introduction of "Responder mode" does not change the IKE SA rekeying behavior: Each node shall be able to operate as initiator and responder in IKE_SA rekeying.

4.6 CMPv2

Certificate Management Protocol version 2 (CMPv2) is based on Internet X.509 Public Key Infrastructure (PKI) Certificate Management Protocol (CMP) specified in IETF RFC 4210 [38], IETF RFC 9480 [95], IETF RFC 9481 [96], IETF RFC 4211 [39], and IETF RFC 9045 [97].

3GPP TS 33.310 [26] specifies the use of CMPv2 used by base stations to obtain an operator-signed certificate using a secured communication based on the vendor-signed certificate in the base station and a vendor root certificate pre-installed in the CA/RA server.

Certificates may be installed at initial system initialization or obtained through certificate enrolment with the operators PKI. Certificate enrolment may be supported with the CMPv2 protocol between the Network Element (NE) and the operators CA as defined in 3GPP TS 33.310 [26]. A PNF that supports CMPv2 shall use the CMPv2 profile defined in 3GPP TS 33.310 [26], clause 9.5. The CA/RA server may include the trust anchor for the operator issued certificate and the appropriate certificate chains in the initialization response message.

4.7 OAuth 2.0

4.7.1 Overview

The authorization framework described in clause 4.7 of the present document uses the OAuth 2.0 framework as specified in IETF RFC 6749 [35]. It allows the service producers to authorize the requests from service consumers, and the service consumers to obtain authorization credentials ("token endpoint").

Interfaces requiring the use of OAuth 2.0 for authorization purposes shall support the functionality according to clause 4.7 of the present document.

Following options have been introduced to guarantee interoperability and align with the existing OAuth 2.0 authorization framework in 3GPP TS 33.501 [29] (i.e., Service Based Architecture).

4.7.2 Basic Parameterization

4.7.2.1 General

Client access token authorization grants shall be supported with type Client Credentials Grant, as described in IETF RFC 6749 [35], section 4.4. Mutual TLS authentication, as specified in the present document (clause 4.2.1), is the mechanism selected by O-RAN for this security procedure.

In addition, still aligned with Client Credential Grant as described in IETF RFC 6749 [35], section 4.4, Client Id and Client Secret may be supported for client authentication. In this case, one-way TLS may be used (certificate on server side).

Access tokens shall be JSON Web Tokens (JWT) as described in IETF RFC 7519 [36], IETF RFC 8725 [93] and IETF RFC 7797 [94], and shall be secured with digital signatures based on JSON Web Signature (JWS) as described in IETF RFC 7515 [37].

The 'scope' attribute as described in IETF RFC 6749 [35], section 3.3, shall be used to specify the allowed services per type of service producer. A more granular level of authorization may be defined by adding additional scope information with the token (e.g., to authorize specific operations, or access to particular resources or datasets), which requires verification by the service producer.

OAuth 2.0 roles, as defined in IETF RFC 6749 [35], section 1.1, are as follows:

- OAuth 2.0 authorization server: new security function in O-RAN architecture; or provided by existing OAuth 2.0 infrastructure in the network.
- OAuth 2.0 client: every API service consumer in O-RAN system (e.g., rApp, xApps)
- OAuth 2.0 resource owner/server: every API service producer in O-RAN system (e.g., Near-RT RIC platform)

4.7.2.2 Registration process

OAuth 2.0 resource servers (API service producers) shall be registered with the OAuth 2.0 authorization server. Service producers may include additional scope information related to specific allowed service operations and resources per client type.

Before initiating the authorization protocol, OAuth 2.0 clients shall be registered with the OAuth 2.0 authorization server. To achieve that, information about the service consumer instance and its type shall be made available in the OAuth 2.0 authorization server. The registration process is subject to implementation procedures of the operator, with the following consideration on authentication procedure:

- OAuth 2.0 clients shall be able to authenticate securely with the authorization server and client type shall be "confidential" as specified in IETF RFC 6749 [35], section 2.1.
- Strong authentication mechanisms based on digital certificates shall be supported.

In addition, a client authentication mechanism based on Client Id and Client Secret may be supported.

4.7.2.3 Access Token request process

4.7.2.3.0 Introduction

After TLS mutual authentication procedure between the OAuth 2.0 client and OAuth 2.0 authorization server, or one-way TLS authentication with server certificate and client authentication based on Client Id and Client Secret has been executed, the authorization server exposes a 'Token Endpoint' where the Access Token request service can be requested by OAuth 2.0 client.

The following procedure depicts the procedure of the OAuth 2.0 client to obtain an access token from the OAuth 2.0 authorization server.

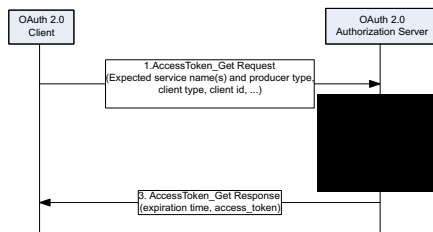


Figure 4.7-1: Access Token request process

The OAuth 2.0 client shall request an access token from the OAuth 2.0 authorization server. For this operation the client shall send an HTTP POST request to the authorization server ('Token endpoint'), as described in IETF RFC 6749 [35], clause 3.2. The message, i.e., the body of the HTTP POST request, shall include the identifier of the client instance making the request, the 'scope' parameter indicating the expected services, and optionally additional scope information with more granular information about resources and operations on the resources, and optionally the type of client ('confidential') and the expected OAuth 2.0 resource owner/server. The message may include information referring to instance(s) of specific resource owner(s)/server(s) as well, if required.

4.7.2.3.1 Server requirements

- The OAuth 2.0 authorization server shall verify that the input parameters in the access token (e.g., client type) match with the corresponding ones in the public certificate of the client, or those in the client profile previously registered.
- If the client is authorized, the authorization server shall generate an access token with appropriate scope as defined in IETF RFC6749[35], section 3.3.
- The authorization server shall digitally sign the generated access token based on a private key as described in IETF RFC 7515 [37]. If the client is not authorized, the Authorization server shall not issue an access token to that client. If the authorization is successful, the Authorization server shall send the access token to the client ('200 OK'), otherwise it shall reply based on OAuth 2.0 error response defined in IETF RFC 6749 [35].
- The success response should include in addition the expiration time for the token as indicated in IETF RFC 6749 [35].
- The response shall include information to identify the resource owner(s)/server(s) if they differ from the related information included in the token request.

4.7.2.4 Service access request based on token verification

Once the client is in possession of a valid access token, it may proceed with the request of service access towards the service producer (OAuth 2.0 resource owner/server).

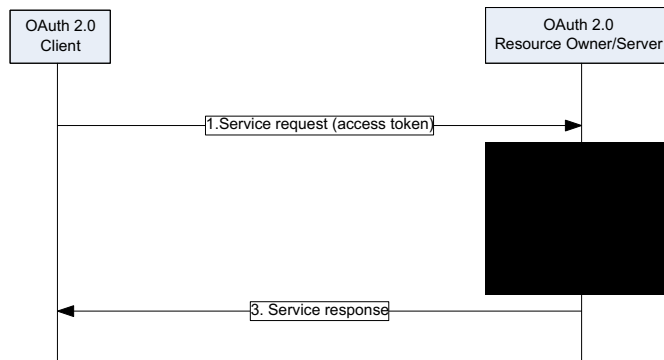


Figure 4.7-2: Service access request based on token verification

1. The OAuth 2.0 client requests service from the OAuth 2.0 resource owner/server. The OAuth 2.0 client shall include the access token in the request. The OAuth 2.0 client and OAuth 2.0 resource owner/server shall authenticate each other via mutual TLS, as defined in the present document (clause 4.2.1) In addition, one-way TLS authentication with certificate only on server side may be supported, as defined in the clause 4.2.1.
2. The OAuth 2.0 resource owner/server shall verify the token:
 - i. It ensures the integrity of the token by verifying the signature using the OAuth 2.0 authorization server's public key.
 - ii. If the integrity check is successful, the OAuth 2.0 resource owner/server shall verify the claims.
3. If the verification is successful, the OAuth 2.0 resource owner/server shall execute the requested service and respond back to the OAuth 2.0 client. Otherwise, it shall send a proper error response. If the HTTP protocol is used, that error response shall be based on the OAuth 2.0 error response defined in IETF RFC 6749 [35].

4.7.3 OAuth2.0 Access token API for O-RAN Elements and interfaces

4.7.3.1 Introduction

This API allows the API Consumer to request an access token based on the procedures for "Access Token request process" as defined in clause 4.7.2.3.

The API version number format shall follow the definitions in 3GPP TS 29.501 [77], clause 4.3.1.1. The MAJOR version field shall be 1, the MINOR version field shall be 0 and the PATCH version field shall be 0. Consequently, the <apiVersion> URI path segment shall be set to "v1".

The API is under development and consequently the API version shall include the pre-release version "alpha.1".

For the general token endpoint for O-RAN RESTful service APIs, the URI structure should follow the provisions specified below, with the <apiName> resource URI variable set to "oauth2".

The resources and methods overview of the access token API are addressed in [Table 4.7-1](#) and shown in Figure 4.7-3.

Deleted: Table 4.7-1

Table 4.7-1: Resources and methods overview of the Access Token API

Resource Name	Resource URI	HTTP method	Service Operation
Access Token	.../token	POST	Access token request

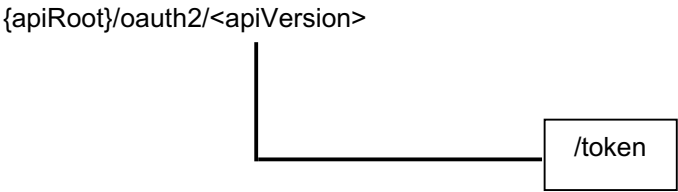


Figure 4.7-3: Resource structure for access token

4.7.3.2 Service operations for access token request

The service operation is defined in IETF RFC 6749 [35] and represented in Figure 4.7-1.

In the present document, only the use of the "Client credentials grant type" (see IETF RFC 6749 [35], sections 1.3.4 and 4.4) is supported.

```
@startuml
!pragma teoz true
skinparam ParticipantPadding 5
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid
autonumber

participant "API Consumer" as AC
participant " Authorization Server " as AS

AC->>AS: POST .../token (AccessTokenReq)
AS->>AS: validate-credentials

alt Success
AS->>AC: 200 Ok (AccessTokenRes)
```

```

else Failure
AS->AC: 400 Bad Request (AccessTokenErr)
end

```

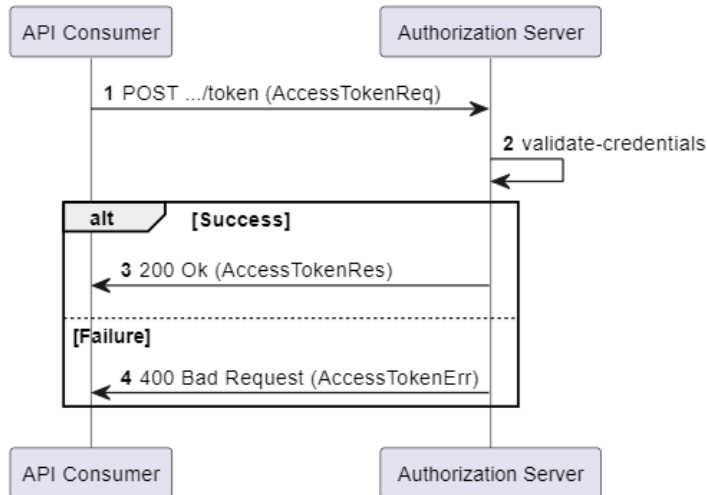


Figure 4.7-4: Access Token Request and Response

1. The API Consumer sends an HTTP POST request to authorization server to request the access token from the Token Endpoint.
2. The authorization server authenticates the client as specified in clause 4.4.2 of IETF RFC 6749 [35] and validates the request as implied in clause 4.4.3 of IETF RFC 6749 [35].
3. On success, the authorization server returns a "200 OK" HTTP response along with an access token response structure.
4. On failure, the authorization server returns a "400 Bad Request" HTTP response and the message content contains additional error information.

4.7.3.2.1 Resource Standard Methods

The POST method and its request and response parameters are specified in IETF RFC 6749 [35].

This information is reproduced using the O-RAN scheme for RESTful API definitions below, with the request data structure represented in Table 4.7.3.2.1-1, the response data structures and response codes represented in Table 4.7.3.2.1-2 and the HTTP headers represented in Table 4.7.3.2.1-3.

Table 4.7-2: Data structures supported by the POST request body on this resource

Data Type	P	Cardinality	Description
AccessTokenReq	M	1	As content specified in IETF RFC 6749 [35]. See also clause 4.7.3.1.2.1. The content type is "application/x-www-form-urlencoded".

Table 4.7-3: Data structures supported by the POST Response Body on this resource.

Data Type	P	Cardinality	Response codes	Description
AccessTokenRsp	M	1	200 OK	As content specified in IETF RFC 6749 [35]. See also clause 4.7.3.1.2.2. The content type is "application/json".
AccessTokenErr	M	1	400 Bad request	As content specified in IETF RFC 6749 [35]. See also clause 4.7.3.1.2.3. The content type is "application/json".

Table 4.7-4: Headers supported by the 200 Response Code on this resource.

Name	Data type	P	Cardinality	Description
Cache-control	string	M	1	As content specified in IETF RFC 6749 [35].
Pragma	string	M	1	As content specified in IETF RFC 6749 [35].

4.7.3.2.2 Data Model

The following clauses represent the message content to be used by the OAuth2.0 access token API as O-RAN data types in line with the provisions in IETF RFC 6749 [35].

4.7.3.2.2.1 Data type: AccessTokenReq

The AccessTokenReq data type represents an access token request to the authorization server. Its attributes are defined by IETF RFC 6749 [35] and reproduced in Table 4.7.3.2.2.1-1.

Table 4.7-5: Definition of type AccessTokenReq

Attribute Name	Data type	P	Cardinality	Description
grant_type	string	M	1	As content specified in IETF RFC 6749 [35], section 4.4.2. In the present document, the only allowed value shall be "client_credentials".
client_id	string	C	0..1	As content specified in IETF RFC 6749 [35]. See NOTE.
client_secret	string	C	0..1	As content specified in IETF RFC 6749 [35]. See NOTE.
scope	string	O	0..1	As content specified in IETF RFC 6749 [35], section 4.4.2.
NOTE: As specified in IETF RFC 6749 [35], section 2.1, there are two options for authenticating the clients to the authorization server: HTTP basic authentication and access token request message content. The attributes "client_id" and "client_secret" shall be absent if HTTP basic authentication is used to convey the client credentials as per IETF RFC6749 [35], section 2.3.1, and shall be present otherwise.				

4.7.3.2.2.2 Data type: AccessTokenRsp

The AccessTokenRsp data type represents an access token response from the authorization server. Its attributes are defined by IETF RFC 6749 [35] and reproduced in table 4.7.3.2.2.2-1.

Table 4.7-6: Definition of type AccessTokenResp.

Attribute Name	Data type	P	Cardinality	Description
access_token	string	M	1	As content specified in IETF RFC 6749 [35], section 5.1.
token_type	string	M	1	As content specified in IETF RFC 6749 [35], section 5.1.
expires_in	integer	O	0..1	As content specified in IETF RFC 6749 [35], section 5.1.
scope	string	O	0..1	As content specified in IETF RFC 6749 [35], section 5.1.

4.7.3.2.2.3 Data type: AccessTokenErr

The AccessTokenErr data type represents an access token error response from the authorization server. Its attributes are defined by IETF RFC 6749 [35] and reproduced in Table 4.7.3.2.2.3-1.

Table 4.7-7: Definition of type AccessTokenErr

Attribute Name	Data type	P	Cardinality	Description
error	string	M	1	As content specified in IETF RFC 6749 [35], section 5.2, the following values are valid: "invalid_request", "invalid_client, invalid_grant", "unauthorized_client", "unsupported_grant_type", "invalid_scope."
error_description	string	O	0..1	As content specified in IETF RFC 6749 [35], section 5.2.
error_uri	string	O	0..1	As content specified in IETF RFC 6749 [35], section 5.2.

4.7.3.3 Service operations for Service access request

The service operation is as follows:

```
@startuml
    !pragma teoz true
    skinparam ParticipantPadding 5
    skinparam BoxPadding 10
    skinparam defaultFontSize 12
    skinparam lifelineStrategy solid
    autonumber

    participant "API Consumer" as AC
    participant " API Producer " as AP

    AC->>AP: GET.../(access_token)
    AP->>AP: validate(access token)

    alt Authorized
        AP->>AC: 200 Ok
    else Access token invalid
        AP->>AC : 401 Unauthorized
    else Insufficient scope
        AP->>AC : 403 Forbidden
    else Other error response
        AP->> AC: 4xx/5xx Error response
    end
end
```

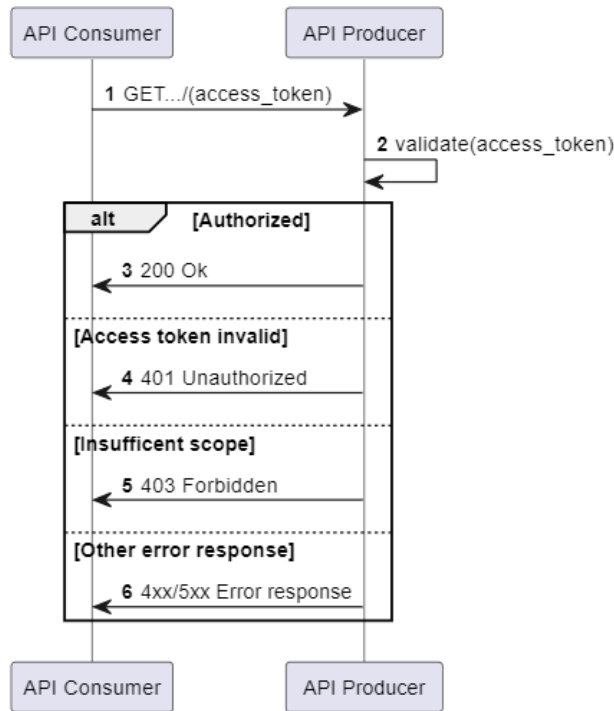


Figure 4.7-5: Service request using access token

1. The API Consumer shall access the protected resources by using access token for the service API being invoked as shown in [Figure 4.7-5](#). The access token shall be included in the request as an HTTP header field as specified in IETF RFC 6750 [78], section 2.1.
2. The API Producer shall validate the access token and ensure that it has not expired and that its scope covers the requested resource.
3. If the token validation is successful, the API Producer shall allow access for the actual resource with the actual request and its parameters. The API Producer shall return the HTTP response as defined for the service API being invoked.
4. If the token validation fails due to invalid token or expired token, the API Producer shall return "401 Unauthorized" in line with the provisions in IETF RFC 6750 [78], section 3.
5. If the token validation is failed due to insufficient access rights to access the resource, the API Producer shall return "403 Forbidden" in line with the provisions in IETF RFC 6750 [78], section 3.
6. In case of any other error, the API Producer shall return the appropriate error as defined by the service API.

Deleted: Figure 4.7-5

4.8 LDAP

4.8.1 General requirements

LDAP, or the Lightweight Directory Access Protocol, is a widely adopted, unbiased standardized communication protocol that enables the interaction and management of distributed directory data via an IP network. This protocol plays a crucial part in the creation of both intranet and internet applications, as it facilitates the sharing of detailed information about various resources, such as users, systems, networks, services, and applications, across the entire network.

O-RAN interfaces that implement Lightweight Directory Access Protocol (LDAP) shall:

- support LDAP version 3 as specified in [74] and LDAP with StartTLS as specified in [75].
- not support previous versions of LDAP.

StartTLS operation shall be initiated prior to any other LDAP operation and all LDAP data shall be sent only after successful establishment of TLS connection. Requirements for TLS in clause 4.2 shall apply.

4.9 MACsec

4.9.1 General requirements

The O-RAN interfaces that implement authentication, confidentiality and integrity using MACsec shall support MACsec capabilities according to IEEE Std 802.1AE-2018 [79].

4.9.2 MACsec Profile

The O-RAN interfaces using MACsec shall support the mandatory cipher suites as specified in IEEE Std 802.1AE-2018 [79] clause 14, Table 14-1—MACsec Cipher Suites.

5 Cryptographic operations

Table 5-1 outlines the main cryptographic operations involved in the protection of (1) application packages for ensuring their integrity, authenticity and confidentiality (for sensitive artifacts) during delivery, onboarding, and instantiation phases, (2) the communication channel over O-RAN interfaces in terms of authenticity, confidentiality and integrity, and (3) the stored assets within O-RAN system. It contains the allowed list of algorithms, key sizes and standards according to BSI [i.6] , NIST [40], [48] and [i.9] cryptographic guidelines.

Deleted: Table 5-1

Table 5-1: Cryptographic Operations for O-RAN

Cryptographic operations	Algorithms	Key sizes	Applicable standards	Usage examples	Note
Signature generation and verification	RSASSA-PSS RSA_PKCS1_V1_5	≥ 2048 -bits	FIPS 186 [42]	For ensuring the integrity and authenticity of application packages during delivery, onboarding, and instantiation phases	Existing code should use PKCS #1 v1.5 padding mode for compatibility only. Implementations may not use null padding.
Signature generation and verification	ECDSA NIST-approved curves (P-256, P-384, or P-521)	≥ 256 -bits	FIPS 186 [42], SP 800-186 [i.8]		Parameter key size for DSA shall not allow the following combination: Bit lengths of L and N parameters L = 2048, N = 224 (BSI TR-02102-1 [i.6])
Symmetric Encryption/Decryption	AES_128, AES_192 and AES_256	128, 192 and 256 bits	FIPS 197 [44], SP 800-38 [i.7] (operation modes)	For ensuring the confidentiality of data	
Asymmetric Encryption/Decryption	RSASSA-OAEP	≥ 2048 -bits	IETF RFC 8017 [50]		
Hashing	SHA-2 family (SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224 and SHA-512/256) SHA-3 family (SHA3-224, SHA3-256, SHA3-384, and SHA3-512)	N/A	FIPS 180 [119] for SHA2 FIPS 202 [47] for SHA 3	For ensuring the integrity of data	
Random Number Generation (RNG)	DRBGs (CTR_DRBG (AES), HMAC_DRBG (SHA-256), Hash_DRBG (SHA-256))	N/A	NIST SP 800-90A [i.12] NIST SP 800-90B [i.13] NIST SP 800-90C [i.14]	For generating cryptographic keys for encryption and digital signatures. For creating random nonces for secure operations.	
	System RNG EXAMPLE: Linux getrandom(), /dev/urandom	N/A	For Linux: BSI [i.16]	For generating unique instance IDs. For providing secure entropy to seed DRBGs.	

The signature operation shall involve X.509-based PKI certificates [74].

The following recommendations should be considered:

- For the protection of cryptographic keys, Hardware Security Modules, or HSMs, should be used.
 - Along with HSMs, the principle of least privilege shall be used to protect keys from unauthorized access.
- In case a legacy system does not support ECDSA, RSA signing algorithms should be used instead. Otherwise, the use of the Elliptic curve signing algorithms is recommended.
- If libraries or frameworks are in use that do not support PSS padding scheme, one of the RSA PKCS1 algorithms should be used instead. Otherwise, the use of one of the RSA PSS algorithms is recommended.
- In general, the largest key size available for an algorithm family should be used:

- For RSA, the largest supported key size is 4096 bits.
- For ECDSA, the largest supported key size is 512 bits.
- For AES, the largest supported key size is 256 bits.

6 Secure File Transfer protocols

6.1 General

File Transfer management is required in several O-RAN use cases, e.g. for export of log files.

An O-RAN architectural element that implements file management shall support at least one of these secure file transfer protocols: SFTP, FTPES and HTTPS. This is aligned with, for example, how 3GPP specifies File Management in [67] and [68].

SFTP is authenticated with username/password, SSH keys or X.509 certificates. FTPES is authenticated with X.509 certificates. HTTPS is mutually authenticated with X.509 certificates.

6.2 SFTP

6.2.1 General Requirements

O-RAN architectural elements that implement secure file transfer using SFTP shall:

- support secure connection and authentication using SSHv2 Authentication Protocol as defined in IETF RFC 4252 [2], IETF RFC 8308 [107], IETF RFC 9519 [80], IETF RFC 8332 [108] with all O-RAN specific requirements from clause 4.1 in the present document;
- support SFTPV3 as the SFTP Industry Best Practice [65].

6.3 FTPES

6.3.1 General Requirements

O-RAN architectural elements that implement secure file transfer using FTPES shall:

- support secure connection and authentication using TLS with all O-RAN specific requirements from clause 4.2 in the present document;
- support FTPES as defined in IETF RFC 4217 [66], and IETF RFC 8996 [112].

6.4 HTTPS

6.4.1 General Requirements

O-RAN architectural elements that implement secure file transfer using HTTPS shall:

- support secure connection and authentication using TLS with all O-RAN specific requirements from clause 4.2 in the present document;
- support HTTPS as defined in [69].

Annex A (normative): OpenAPI specifications

A.1 OAuth2.0 API

A.1.1 Introduction

The Open API for the OAuth2.0 Access Token API specified in clause A.1.2 is based on the API definitions in clause 4.7.3.4.

A.1.2 OAuth 2.0 Access Token API

```
openapi: 3.0.3
info:
  title: OAuth 2.0 Access Token API
  version: 1.0.0
  description: |
    API for OAuth 2.0.
    © 2024, O-RAN ALLIANCE.
    All rights reserved.
externalDocs:
  description: O-RAN.WG11.SecProtSpecs-R003-v10.00
  url: https://www.o-ran.org/specifications
servers:
- url: '{apiRoot}/oauth2/v1'
  variables:
    apiRoot:
      description: apiRoot as defined in in clause 4.4.1 of 3GPP TS 29.501
      default: https://example.com
paths:
  /token:
    post:
      description: Token endpoint
      operationId: AccessTokenRequest
      tags:
        - Access Token Request
      requestBody:
        required: true
        content:
          application/x-www-form-urlencoded:
            schema:
              $ref: '#/components/schemas/AccessTokenReq'
      responses:
        '200':
          description: Success case 200 OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/AccessTokenResp'
          headers:
            Cache-Control:
              $ref: '#/components/headers/cache-control'
            Pragma:
              $ref: '#/components/headers/pragma'
        '400':
          description: Error in the Access Token Request
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/AccessTokenErr'
components:
  schemas:
    AccessTokenReq:
      description: Contains information related to the access token request
      type: object
      properties:
        grant_type:
          description: Represents the Client Credentials grant type
```

```

enum: [ client_credentials ]
type: string
client_id:
  description: >-
    Represents the identity assigned to the client configured in the
    authorization server
  type: string
client_secret:
  description: >-
    Represents the secret assigned to the client configured in the
    authorization server
  type: string
scope:
  description: Represents the scope of the access request
  type: string
required:
  - grant_type
AccessTokenResp:
  description: Contains information related to the access token response
  type: object
  properties:
    access_token:
      description: Represents the access token issued by the authorization server
      type: string
    token_type:
      $ref: '#/components/schemas/token_type'
    expires_in:
      description: Represents the lifetime in seconds of the access token
      type: integer
    scope:
      description: Represents the scope of the access request
      type: string
  required:
    - access_token
    - token_type
token_type:
  description: As described in clause 6.1.1 of IETF RFC 6750
  type: string
enum:
  - Bearer
AccessTokenErr:
  description: Error returned in the access token response message
  type: object
  properties:
    error:
      type: string
      enum:
        - unauthorized_client
        - unsupported_grant_type
        - invalid_request
        - invalid_scope
        - invalid_client
        - invalid_grant
    error_description:
      type: string
    error_uri:
      type: string
  required:
    - error
headers:
  cache-control:
    required: true
  schema:
    type: string
    enum:
      - no-store
pragma:
  required: true
  schema:
    type: string
    enum:
      - no-cache

```

Annex B (informative): Example of TLS entity Certificate Profile for NETCONF authentication

The purpose of the Annex is to provide an example of a TLS entity certificate profile which is used for NETCONF authentication.

As specified in clause 6.4.2 of the O-RAN Working Group 4 Management Plane Specification [6], if authentication is based on X.509 certificates, then for the purposes of user authentication, the mapping between certificates and **user-name** is provided by the subjectAltName field of the X.509 certificate, which means that the user name is coded in the subjectAltName. The username is determined from the subjectAltName using the rules as specified in RFC 7589 [i.11]. For the purposes of NETCONF server authentication, RFC 7589 [i.11] specifies server identity as specified in clause 4 of RFC 9525 [98]. An example of a default user account for account-type CERTIFICATE is map type "san-rfc822-name" with a rfc822-name of "oranuser@o-ran.org".

Table B-1

O-RAN TLS Certificate Profile			
Field		Value	
version		3	
serial Number		Unique Positive Integer (up to 20 octets) assigned by the issuing CA as per RFC 5280 [64].	
issuer		EXAMPLE: (C=<Country>), O=<Organization Name>, CN=< Some distinguishing name>.	
subject		EXAMPLE: (C=<Country>), O=<Organization Name>, CN=<Common Name such as server domain name>.	
validity		1 year or less	
signature		See 3GPP TS 33.310 [26], clause 6.1.1 for the list of supported signature algorithms.	
subjectPublicKey Info		See 3GPP TS 33.310 [26], clause 6.1.1 for the list of supported public key types.	
Extensions	Mandatory	Criticality	Value
keyUsage	TRUE	TRUE	digitalSignature for TLS clients and servers
extendedKeyUsage	FALSE	FALSE	id-kp-clientAuth TLS clients
			id-kp-serverAuth for TLS servers
			Entities that may be both client and server will have both OIDs set.

authorityKeyIdentifier	FALSE	FALSE	This is same as the subjectKeyIdentifier of the issuer's certificate. CA utilizes the method as defined in clause 2 of IETF RFC 7093 [60].
subjectKeyIdentifier	FALSE	FALSE	This is calculated by the issuing CA utilizing method as defined in clause 2 of IETF RFC 7093 [60].
subjectAltName	FALSE	FALSE	rfc822Name
cRLDistributionPoint	TRUE	FALSE	distributionPoint
			According to IETF RFC 5280 [64] this indicates if the CRL is available for retrieval using access protocol and location with HTTP URI or LDAP.
authorityInfoAccess	FALSE	FALSE	id-ad-caIssuers
			According to IETF RFC 5280 [64] id-ad-caIssuers describes the referenced description server and the access protocol and location, for example, using one or multiple HTTP and/or LDAP URIs. The referenced CA issuers description is intended to aid certificate users in the selection of a certification path that terminates at a point trusted by the certificate user
			id-ad-ocsp
			According to IETF RFC 5280 [64] id-ad-ocsp defines the location of the OCSP responder using HTTP URI.

Annex (informative): Change history/Change request (history)

Date	Revision	Description
2025.07.11	13.00	Published as Final version 13.00
2025.07.10	12.01	Added SYM-0086, ATT.AO-5006, NOK-0291 and ERI-0230; fixed editorial errors
2025.03.23	12.00	More editorial fixes; Published as Final version 12.00
2025.03.21	11.03	Added ACN.AO-0006, RMI.AO-0035, ACN-0009, ACN.AO-0007, SYM-0083, NOK-0246, ACN-0010, NOK.AO-0257, NOK-0259, NOK-0264, NOK-0265, ACN-0012, ACN-0013, ACN.AO-0014, ACN.AO-0015
2025.03.19	11.02	Updated formatting to align with latest TS template (v3.0)
2025.02.18	11.01	Updated formatting, added NOK-0232, NOK-0233
2024.11.27	11.00	More editorial fixes Published as Final version 11.00
2024.11.27	10.04	Disabled automatic update of styles in the document developer settings
2024.11.27	10.03	Further editorial changes
2024.11.27	10.02	Several minor editorial- and format corrections
2024.11.27	10.01	Added DTAG-0013, ATT.AO-0061, ACN.AO-0004 and ERI-0172
2024.07	10.00	Published as Final version 10.00
2024.03	09.00	Certificate Profile for O-RAN TLS entity Reference Update Published as Final version 09.00
2023.11	08.00	Secure file transfer protocols added SSH Update on no root remote login ssh-ed448 changed to optional Reference on SSH added Published as Final version 08.00
2023.07	07.00	Editorial alignments TLS entity certificate profile CMPv2 profile update Introduction of one-way TLS authentication for OAuth2.0 Published as Final version 07.00
2023.03	06.00	Cryptographic operations update Published as Final version 06.00
2022.11	05.00	TLS Cipher update Published as Final version 05.00
2022.07.20	04.00	Addition of CMPv2 Update of O-Cloud Image protocols Addition of mTLS Update of OAuth 2.0 Published as Final version 04.00
2021.11.08	03.00	Update the O-RAN security protocols and specifications to include mandatory support for TLS 1.3 Published as Final version 03.00
2021.07.05	02.00	Addition of DTLS and IPsec requirements. Alignment of TLS 1.2 and TLS 1.3 profiles with 3GPP TS 33.210. Typographical changes. Published as Final version 02.00
2021.04.01	01.00	Initial version of the document with requirements for TLS and SSH. Published as Final version 01.00